

Cas d'utilisation : Inscrire un participant à un cours

Acteur

Administrateur du studio / Utilisateur

Objectif

Permettre d'associer un participant existant à un cours existant afin d'enregistrer son inscription.

Données nécessaires

L'utilisateur fournit :

Champ	Description
Id_Participant	Identifiant du participant
Id_Cours	Identifiant du cours

Traitement

1. L'utilisateur sélectionne un participant.
2. L'utilisateur sélectionne un cours.
3. L'API vérifie :
 - Que le participant existe.
 - Que le cours existe.
 - Que le cours n'est pas complet.
 - Que le participant n'est pas déjà inscrit à ce cours.

4. Création d'une inscription dans la table **INSCRIPTION**.
5. Mise à jour du nombre d'inscrits du cours (si ce champ est conservé).

Diagramme de séquence simplifié

```
sequenceDiagram
    participant Utilisateur
    participant API as API Inscription
    participant Base as Base de données
    Utilisateur->>API: | Id participant + Id cours
    API->>Base: | Vérification participant
    API->>Base: | Vérification cours
    API->>Base: | Vérification places disponibles
    Base-->>API: | INSERT INSCRIPTION
    Base-->>API: | UPDATE COURS
    API-->>Utilisateur: Inscription validée
```

Utilisateur

|

| Id participant + Id cours

v

API Inscription

|

| Vérification participant

|

| Vérification cours

|

| Vérification places disponibles

v

Base de données

|

| INSERT INSCRIPTION

|

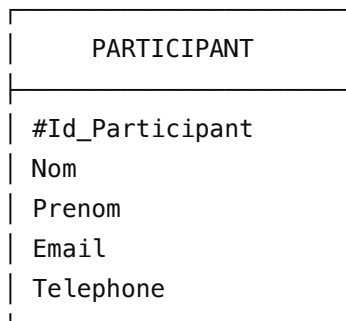
| UPDATE COURS

v

Inscription validée

Entités concernées dans le MCD

PARTICIPANT



INSCRIPTION

INSCRIPTION
#Id_Inscription
Date_Inscription
Statut
Id_Participant (FK)
Id_Cours (FK)



COURS

COURS
#Id_Cours
Nom_Cours
Nom_Coach
Capacite_Max
Prix



Règles de gestion

RG1 : Un participant peut être inscrit à plusieurs cours.

PARTICIPANT (0,N) ---- INSCRIPTION



RG2 : Un cours peut avoir plusieurs participants.

COURS (0,N) ---- INSCRIPTION



RG3 : Une inscription appartient obligatoirement à un seul participant et un seul cours.

INSCRIPTION (1,1)



RG4 : Le nombre d'inscriptions ne doit pas dépasser la capacité du cours.

Exemple :

Capacité du cours : 12
Inscrits actuels : 12



Résultat :
Inscription impossible – cours complet

RG5 : Un participant ne peut pas être inscrit deux fois au même cours.

Endpoint API

Inscrire un participant

</> http



≡ ChatGPT ▾ **envoyé** Mettre à niveau



```
{  
  "id_participant": 1,
```

</> JSON



Réponse succès

Code HTTP :

</> http



201 Created

</> JSON



```
{  
  "message": "Inscription validée",  
  "inscription": {  
    "id_inscription": 15,  
    "participant": "Marie Dupont",  
    "cours": "Yoga débutant",  
    "date_inscription": "2026-06-20",  
    "statut": "Confirmée"
```

```
}  
}
```

Réponses d'erreur possibles

Participant inexistant

```
</> JSON  
  
{  
  "message": "Participant introuvable"  
}
```



Cours inexistant

```
</> JSON  
  
{  
  "message": "Cours introuvable"  
}
```



Cours complet

```
</> JSON  
  
{  
  "message": "Impossible : ce cours est complet"  
}
```



Déjà inscrit

```
</> JSON  
  
{  
  "message": "Ce participant est déjà inscrit à ce cours"  
}
```



Cette fonctionnalité utilise pleinement ton MCD :

PARTICIPANT → INSCRIPTION ← COURS

et représente l'opération centrale de ton API de réservation.



Compter les places disponibles

Cas d'utilisation : Compter les places disponibles

Acteur

Utilisateur / Administrateur du studio

Objectif

Permettre de connaître le nombre de places restantes pour un cours collectif avant d'effectuer une inscription.

Données nécessaires

L'utilisateur fournit :

Champ

Description

Id_Cours

Identifiant du cours à consulter

Traitement

1. L'utilisateur sélectionne un cours.
2. L'API recherche le cours dans la base de données.
3. L'API récupère :
 - La capacité maximale du cours.
 - Le nombre d'inscriptions enregistrées.
4. L'API calcule le nombre de places disponibles.

Formule :

Places disponibles = Capacite_Max - Nombre_Inscrits



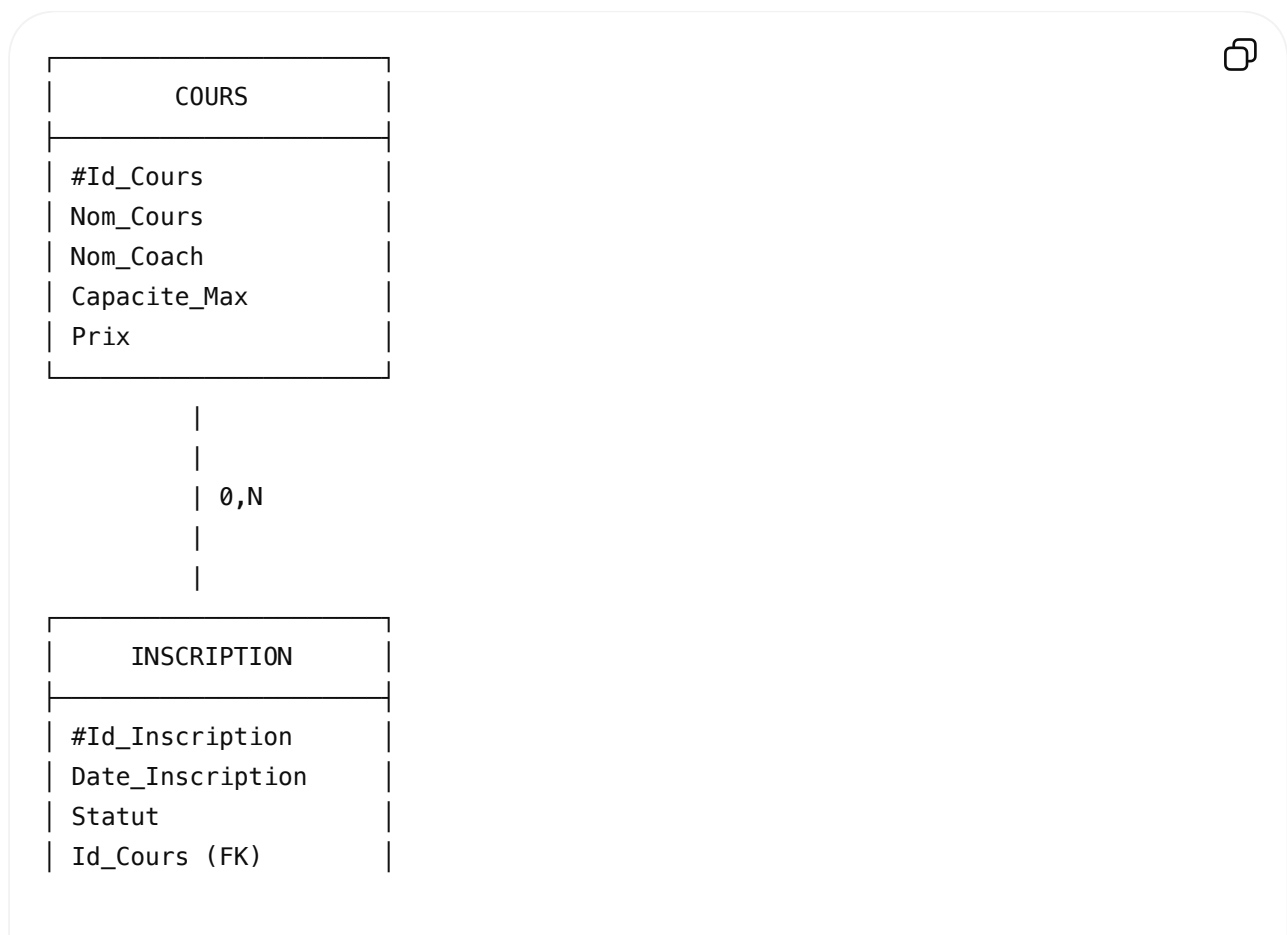
Le nombre d'inscrits est obtenu grâce à la table **INSCRIPTION**.

Diagramme de séquence simplifié



Entités concernées dans le MCD

COURS



| Id_Participant (FK) |

Règles de gestion

RG1 : Un cours possède une capacité maximale définie lors de sa création.

RG2 : Le nombre d'inscriptions correspond au nombre de participants inscrits au cours.

Nombre_Inscrits = COUNT(INSCRIPTION)



RG3 : Les places restantes sont calculées automatiquement.

Places_Disponibles = Capacite_Max - COUNT(INSCRIPTION)



RG4 : Un cours est complet lorsque :

COUNT(INSCRIPTION) >= Capacite_Max



Exemple

Cours :

Information	Valeur
Nom	Yoga débutant
Capacité maximale	12
Participants inscrits	8

Calcul :

12 - 8 = 4



Résultat :

Places disponibles : 4



Endpoint API

Consulter les places disponibles d'un cours

</> http



GET /courses/{id}/places

Exemple :

</> http



GET /courses/1/places

Réponse API

</> JSON



```
{
  "cours": "Yoga débutant",
  "capacite_max": 12,
  "places_occupees": 8,
  "places_disponibles": 4,
  "statut": "Places disponibles"
}
```

Cas où le cours est complet

Réponse :

</> JSON



```
{
  "cours": "Pilates",
  "capacite_max": 10,
  "places_occupees": 10,
  "places_disponibles": 0,
  "statut": "Complet"
}
```

Cette fonctionnalité repose sur la relation :

COURS (0,N) ----- (1,1) INSCRIPTION



Elle permet à l'API de gérer automatiquement la disponibilité des cours avant toute nouvelle inscription.



Cas d'utilisation : Afficher les cours complets

Acteur

Utilisateur / Administrateur du studio

Objectif

Permettre d'afficher uniquement les cours dont le nombre d'inscriptions a atteint la capacité maximale autorisée.

Données nécessaires

Aucune donnée obligatoire n'est fournie par l'utilisateur.

L'API recherche automatiquement tous les cours dans la base de données.

Traitement

1. L'utilisateur demande la liste des cours complets.
2. L'API récupère tous les cours enregistrés.
3. Pour chaque cours, elle compte le nombre d'inscriptions associées.
4. Elle compare :

Nombre d'inscriptions >= Capacité_Max



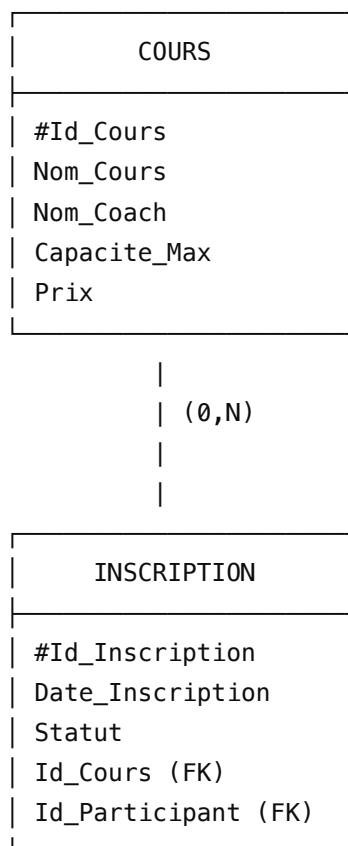
5. Les cours correspondant à cette condition sont retournés.

Diagramme de séquence simplifié

Utilisateur
|
| Demande des cours complets
v
API Cours
|
| Recherche des cours
v
Base de données
|
| Lecture COURS
|
| Comptage INSCRIPTION
v
Vérification capacité atteinte
|
v
Liste des cours complets affichée



Entités concernées dans le MCD



Règles de gestion

RG1 : Un cours est considéré comme complet lorsque :

Nombre d'inscriptions = Capacite_Max



ou

COUNT(INSCRIPTION) >= Capacite_Max



RG2 : Seuls les cours complets doivent être affichés.

RG3 : Si aucun cours n'est complet, l'API retourne un message :

Aucun cours complet trouvé.



Exemple de données

Cours disponibles

Id	Cours	Capacité	Inscrits
1	Yoga débutant	12	8
2	Pilates	10	10
3	Fitness	15	15

Résultat affiché

Cours complets :



#2 – Pilates

Coach : Nora

Places : 10 / 10

#3 – Fitness

Coach : Marc

Places : 15 / 15

Endpoint API

Récupérer les cours complets

</> http

GET /courses/full



Réponse API

</> JSON

```
[
  {
    "id_cours": 2,
    "nom_cours": "Pilates",
    "nom_coach": "Nora",
    "capacite_max": 10,
    "inscrits": 10,
    "statut": "Complet"
  },
  {
    "id_cours": 3,
    "nom_cours": "Fitness",
    "nom_coach": "Marc",
    "capacite_max": 15,
    "inscrits": 15,
    "statut": "Complet"
  }
]
```



Requête SQL correspondante

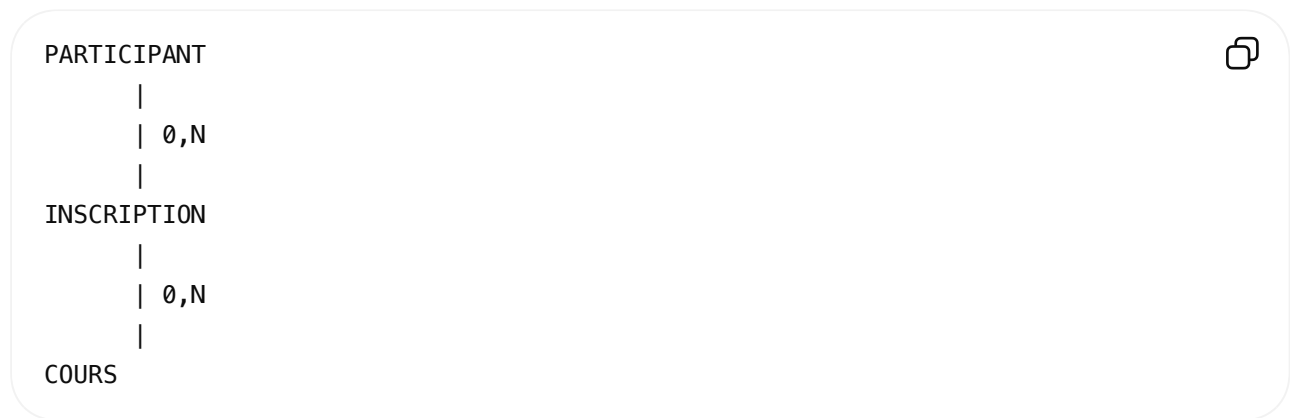
</> SQL

```
SELECT
  cours.id_cours,
  cours.nom_cours,
  cours.capacite_max,
  COUNT(inscription.id_inscription) AS inscrits
FROM cours
JOIN inscription
ON cours.id_cours = inscription.id_cours
```



```
GROUP BY cours.id_cours
HAVING COUNT(inscription.id_inscription) >= cours.capacite_max;
```

Cette fonctionnalité utilise la relation :



et permet au studio de connaître rapidement les cours qui ne peuvent plus accepter de nouveaux participants.



Supprimer un cours

Cas d'utilisation : Supprimer un cours

Acteur

Administrateur du studio

Objectif

Permettre à l'administrateur de supprimer un cours existant de la base de données.

Données nécessaires

L'utilisateur fournit :

Champ	Description
Id_Cours	Identifiant unique du cours à supprimer

Traitement

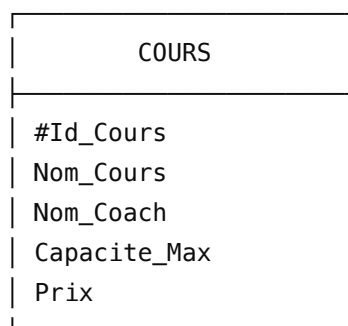
1. L'administrateur sélectionne le cours à supprimer.
2. L'API recherche le cours grâce à son identifiant.
3. L'API vérifie que le cours existe.
4. Suppression des inscriptions associées au cours.
5. Suppression du cours dans la table **COURS**.
6. Retour d'un message de confirmation.

Diagramme de séquence simplifié

```
sequenceDiagram
    participant Administrateur
    participant API Cours
    participant Base de données
    Administrateur->>API Cours: Id_Cours
    API Cours->>Base de données: Recherche du cours
    Base de données->>Base de données: Vérification existence
    Base de données->>Base de données: Suppression INSCRIPTION
    Base de données->>Base de données: DELETE COURS
    Base de données-->>API Cours: Cours supprimé avec succès
```

Entités concernées dans le MCD

La suppression concerne principalement l'entité **COURS**.



| 0,N

|

|

INSCRIPTION

#Id_Inscription

Date_Inscription

Statut

Id_Cours (FK)

Id_Participant (FK)

Règles de gestion

RG1 : Un cours ne peut être supprimé que s'il existe.

Si Id_Cours trouvé → suppression
Sinon → "Cours introuvable"



RG2 : Lorsqu'un cours est supprimé, ses inscriptions doivent également être supprimées.

Relation concernée :

COURS (1) ----- (0,N) INSCRIPTION



Suppression en cascade :

Suppression COURS

↓

Suppression INSCRIPTION associées



RG3 : Les participants ne sont pas supprimés.

Exemple :

Avant :

Participant : Marie Dupont



Cours :

- Yoga
- Pilates

Inscription :

Marie → Yoga
Marie → Pilates

Suppression du cours Yoga :

Participant :

Marie Dupont (conservée)



Cours :

- Pilates

Inscription :

Marie → Pilates

Endpoint API

Supprimer un cours

</> http

DELETE /courses/{id}



Exemple :

</> http

DELETE /courses/2



Réponse succès

Code HTTP :

</> http

200 OK



</> JSON



```
{  
  "message": "Cours supprimé avec succès"  
}
```

Réponse erreur

Si le cours n'existe pas :

Code HTTP :

```
</> http  
  
404 Not Found
```



```
</> JSON  
  
{  
  "message": "Cours introuvable"  
}
```



Exemple

Avant suppression :

Id_Cours	Nom	Coach
1	Yoga débutant	Emma
2	Pilates	Nora

Commande :

```
</> http  
  
DELETE /courses/2
```



Après suppression :

Id_Cours	Nom	Coach
1	Yoga débutant	Emma

Cette fonctionnalité complète le CRUD de l'API :

- ✓ Créer un cours
- ✓ Afficher les cours
- ✓ Inscrire un participant
- ✓ Compter les places disponibles
- ✓ Afficher les cours complets
- ✓ Supprimer un cours



Garder l'historique des inscriptions

Cas d'utilisation : Garder l'historique des inscriptions

Acteur

Administrateur du studio / Utilisateur

Objectif

Conserver une trace de toutes les inscriptions effectuées afin de connaître l'historique des participants, des cours suivis et des dates d'inscription.

Principe

Au lieu de supprimer définitivement une inscription lorsqu'un cours est supprimé ou lorsqu'une inscription est annulée, l'application conserve les informations dans la table **INSCRIPTION**.

L'historique permet de répondre aux questions :

- Qui a participé à quel cours ?
- À quelle date la personne s'est-elle inscrite ?
- Quel était le statut de l'inscription ?
- Combien d'inscriptions ont été réalisées sur un cours ?

Entités concernées dans le MCD

PARTICIPANT

PARTICIPANT
#Id_Participant
Nom
Prenom
Email
Telephone



INSCRIPTION (historique)

INSCRIPTION
#Id_Inscription
Date_Inscription
Statut
Date_Annulation
Id_Participant (FK)
Id_Cours (FK)



COURS

COURS
#Id_Cours
Nom_Cours
Nom_Coach
Capacite_Max
Prix

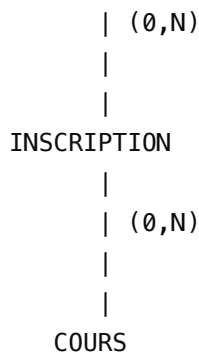


Relation MCD

PARTICIPANT

|





Règles de gestion

RG1 : Une inscription est conservée après sa création.

Exemple :

Marie Dupont
→ Yoga débutant
→ Inscription le 15/06/2026
→ Statut : Confirmée



Cette information reste enregistrée.

RG2 : Une inscription possède un statut.

Valeurs possibles :

Statut	Signification
Confirmée	Participant inscrit au cours
Annulée	Inscription annulée mais conservée
Terminée	Cours réalisé

RG3 : La suppression d'un cours ne supprime pas forcément l'historique.

Deux solutions possibles :

Solution recommandée : suppression logique

On ajoute un champ :

Actif



dans la table COURS.

Exemple :

Avant :

Cours : Yoga débutant
Actif : Oui



Après suppression :

Cours : Yoga débutant
Actif : Non



L'historique reste visible.

RG4 : Le participant n'est jamais supprimé automatiquement.

Même si un cours disparaît :

Participant :
Marie Dupont



reste enregistré.

Exemple d'historique

Requête :

```
</> http  
GET /participants/1/history
```



Réponse :

```
</> JSON  
  
{  
  "participant": "Marie Dupont",  
  "historique": [  
    {
```



```
"cours": "Yoga débutant",
"date_inscription": "2026-06-15",
"statut": "Terminée"
},
{
  "cours": "Pilates",
  "date_inscription": "2026-06-20",
  "statut": "Confirmée"
}
]
}
```

Nouvelle structure de base de données (MLD)

PARTICIPANT

id_participant PK
nom
prenom
email
telephone

COURS

id_cours PK
nom_cours
nom_coach
capacite_max
prix
actif

INSCRIPTION

id_inscription PK
id_participant FK
id_cours FK
date_inscription
statut
date_annulation



Avantages

- ✓ Conservation de l'historique client
- ✓ Suivi des participations passées
- ✓ Statistiques possibles (cours populaires, chiffre d'affaires)
- ✓ Respect d'une logique professionnelle de réservation
- ✓ Compatible avec FastAPI + SQLAlchemy + SQLite

Cette évolution rend ton MCD proche d'une vraie application de gestion de studio sportif.

